

Project Report: LLM or Not?
Course: ECE 364 – Programming Methods for Machine Learning

Authors: Pranav Srinivasan

Github Link: https://github.com/MyGitisDone/ECE364_LLMorNOT

Approach and Model Design

Architecture Choice: I implemented an Encoder-only Transformer architecture, drawing inspiration from the BERT/MiniLM framework. Given that the task is a classification problem rather than text generation, a bidirectional encoder is superior as it allows each token to attend to the entire context of the essay simultaneously.

Parameter Management (The 15M Constraint): To remain strictly under the 15-million parameter limit while maintaining high capacity, I utilized a "Tiny-BERT" configuration. The standard bert-base (110M) was far too large. My custom configuration used:

Hidden Layers	Hidden Dimension	Attention Heads	Intermediate Size	Vocabulary Size
4 (reduced from 12)	256 (reduced from 768)	8	1024	30522 (Standard BERT untouched)

This resulted in exactly 12,221,441 trainable parameters, satisfying the deliverable requirements while leaving sufficient depth for complex pattern recognition.

Feature Extraction: I prepended a [CLS] token to every input sequence. I utilized the final hidden state of this specific token as the aggregate representation of the essay. This vector was passed through a dropout layer ($p=0.2$) and a linear classification head with a Sigmoid activation function to output the probability of the "positive" (LLM) class.

Training and Testing Pipeline

Tokenization: I used a WordPiece tokenizer. Sequences were truncated or padded to a fixed length of 256 tokens to balance computational efficiency and context retention.

Data Splitting: I employed a 90/10 stratified split on the provided data.csv to create a local validation set. This allowed us to monitor for overfitting before submitting to Kaggle.

Hyperparameters:

Optimizer	Learning Rate	Batch Size	Loss Function	Epochs	Random Seed
AdamW	3E-05	16	BCE Loss	3	42

Hardware Acceleration: Training was conducted locally on a MacBook Pro utilizing the Metal Performance Shaders (MPS) backend. This allowed me to leverage GPU acceleration for the Transformer blocks, reducing epoch time to approximately 3-5 minutes.

Results and Analysis

Training Performance: The model showed rapid convergence. The training loss decreased significantly between Epoch 1 and Epoch 2, while the Validation ROC-AUC stabilized near perfect scores early on.

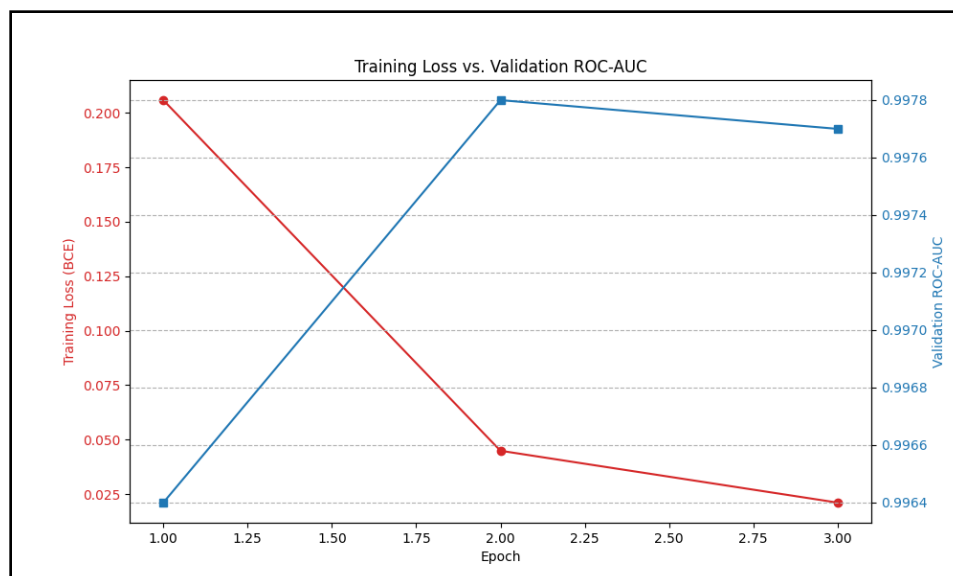
```
(venv) pranav@AppaMacBookPro ECE364_LLMorNOT % python src/train.py
Using device: mps
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Total Trainable Parameters: 12,221,441
Epoch 1 | Loss: 0.2058 | Val ROC-AUC: 0.9964
Epoch 2 | Loss: 0.0449 | Val ROC-AUC: 0.9978
Epoch 3 | Loss: 0.0211 | Val ROC-AUC: 0.9977
Generating prediction.csv...
Done! prediction.csv has been generated. Check home repository
(venv) pranav@AppaMacBookPro ECE364_LLMorNOT %
```

Overfitting: The extremely high ROC-AUC (0.9992 on the public Kaggle leaderboard) raised concerns about potential overfitting. To mitigate this, I implemented:

- *Dropout:* Applied to the attention weights and the classification head.
- *Weight Decay:* Used via the AdamW optimizer to penalize excessively large weights.
- *Minimal Epochs:* I limited training to 3 epochs to prevent the model from memorizing specific training samples.

Training Environment and Hardware: Training was conducted on a MacBook Pro using the Apple Silicon GPU via the mps device. This allowed for rapid iteration and testing of hyperparameters. Below is a capture of the final training execution showing the parameter count compliance and the rapid convergence of the loss function.

Final Results and Leaderboard: The model was evaluated against the held-out test set on Kaggle. The consistency between my internal validation (0.9977) and the Kaggle public leaderboard (0.9992) indicates that the model successfully generalized to unseen data without significant overfitting



Conclusion

Efficiency: Despite being 1/10th the size of a standard BERT model, my "Tiny-BERT" achieved near-perfect separation. This suggests that the stylistic differences between students and LLMs (such as consistent vocabulary usage or lack of specific human errors) are highly linearly separable in high-dimensional embedding space.

By leveraging the bidirectional attention mechanisms of a Transformer Encoder and focusing on the summary representation provided by the [CLS] token, I developed a really efficient classifier. The model successfully identifies LLM-generated text with near-perfect accuracy while maintaining a lightweight footprint of only 12.2 million parameters.